Supplementary Material: A Differentiable Atari VCS: A Complex, Fully Known Ground Truth for Explainable AI

Anonymous submission

This supplement gives the full proofs of the two theorems stated in the main paper, together with the implementation-effort plot referenced in the Results. It is self-contained: we first restate the soft/hard formulation, then prove the forward-equivalence and temperature-limit results, and finally show the effort timeline.

1 Setup and Notation

The differentiable emulator runs one of two transition maps over the full VCS state $x = (\text{registers}, \text{RAM}, \text{RIOT}, \text{TIA}, \text{frame buffer})$: the bit-exact *hard* map Φ_{H} and the differentiable *soft* map Φ_{S}. In both, the handler that runs is the one for the opcode byte at the program counter pc. The handlers are assembled from four primitives, which we restate here so that the proofs are self-contained.

The first primitive is the memory read. A read of a memory tensor $r \in \mathbb{R}^M$ of size M (the ROM or the RAM) at an integer address $a \in \{0, \dots, M-1\}$ is written as a dot product against the one-hot address indicator $\mathbf{1}_a$,

$$\mathrm{peek}(r, a) = \mathbf{1}_a^\top r = \textstyle\sum_i [\![i = a]\!]\, r_i = r_a, \tag{1}$$

so the forward value is the array element r_a, while the gradient with respect to r is the one-hot vector $\mathbf{1}_a$.

The second primitive is opcode dispatch. With a score vector (logits) $\ell \in \mathbb{R}^K$ over the $K = 256$ opcodes, a temperature $T > 0$, and the softmax weights $w = \mathrm{softmax}(\ell/T)$, the relaxed dispatch over the candidate handler-output rows V is the convex combination

$$\mathrm{select}(\ell, V; T) = w^\top V, \qquad w_k = \frac{e^{\ell_k/T}}{\sum_j e^{\ell_j/T}}. \tag{2}$$

The third primitive is the conditional branch. For a relaxed status flag $f \in [0, 1]$ written as a logit $z = s(2f - 1)$, where the sign s selects branch-when-set or branch-when-clear, and a sharpness α, the gate and the relaxed program counter are

$$g = \sigma(\alpha z), \qquad \mathrm{pc}_{\mathrm{soft}} = (1 - g)\,\mathrm{pc}_{\bar{b}} + g\,\mathrm{pc}_b = \mathrm{pc}_{\bar{b}} + g\,\delta, \tag{3}$$

where $\mathrm{pc}_{\bar{b}}$ is the fall-through (not-taken) counter and $\mathrm{pc}_b = \mathrm{pc}_{\bar{b}} + \delta$ is the taken counter for the signed branch offset δ.

The fourth primitive is the straight-through estimator. For any soft value that has a matching hard value it returns

$$\mathrm{STE}(\text{soft}, \text{hard}) = \text{soft} + \mathrm{sg}(\text{hard} - \text{soft}), \tag{4}$$

where $\mathrm{sg}(\cdot)$ is the stop-gradient operator, so the forward value equals *hard* while the backward pass uses the gradient of *soft*.

In the executed soft step, dispatch uses the saturated (hard) form of (2) and the branch returns $\mathrm{STE}(\mathrm{pc}_{\mathrm{soft}}, \mathrm{pc}_{\mathrm{hard}})$. The relaxed forms (2) and (3) *without* the straight-through correction define the "fully relaxed" variant analysed in Theorem 2. The temperature-limit result needs one non-degeneracy assumption, identical to the one in the main paper.

Assumption 1 (Decision margins)**.** *Along the executed trajectory of length N, every discrete decision has a strictly positive margin. For opcode dispatch at step t with logits $\ell^{(t)}$ and winner $k_t^\star$, the logit gap $\Delta_t = \ell^{(t)}_{k_t^\star} - \max_{k \neq k_t^\star} \ell^{(t)}_k$ is positive, and for each conditional branch with flag logit z_t, the margin $m_t = |z_t|$ is positive. Let $\Delta_{\min} = \min_t \Delta_t > 0$ and $m_{\min} = \min_t m_t > 0$.*

2 Proofs

Theorem 1 (Exact forward equivalence)**.** *For every reachable state x and every finite sharpness α and temperature T, the soft and hard one-step maps are equal by design, $\Phi_{\mathrm{S}}(x) = \Phi_{\mathrm{H}}(x)$, with identical float32 representations. Consequently, over a trajectory of N steps, $x_t^{\mathrm{S}} = x_t^{\mathrm{H}}$ for all $t \le N$. The modes differ only in the Jacobian $\partial\Phi_{\mathrm{S}}/\partial x$, which is defined and generically nonzero where $\partial\Phi_{\mathrm{H}}/\partial x$ is zero or undefined.*

Proof. We first prove the one-step equality $\Phi_{\mathrm{S}}(x) = \Phi_{\mathrm{H}}(x)$ for an arbitrary reachable x, and then extend it to a trajectory of length N by induction on the step index.

One-step equality. Fix a reachable x. In both modes the handler that executes is selected by the decoded opcode byte through the same hard switch (the softmax form (2) is not evaluated on the forward path), so both modes run the same handler on the same input. It therefore suffices to show that each primitive the handler uses returns the same forward value in both modes. A memory read at the integer address

a uses an exact one-hot indicator, so

$$\mathrm{peek}_{\mathrm{S}}(r,a) = \mathbf{1}_a^\top r = \textstyle\sum_i [\![i = a]\!]\, r_i = r_a = \mathrm{peek}_{\mathrm{H}}(r,a), \tag{5}$$

the value a hard array read returns. Register, status-flag, binary-coded-decimal, and timer updates use the same integer and round arithmetic in both modes (soft mode merely keeps the results in float32). A conditional branch returns the straight-through value (4). Since sg is the identity in the forward direction,

$$\begin{aligned}\mathrm{STE}(\mathrm{pc}_{\mathrm{soft}}, \mathrm{pc}_{\mathrm{hard}}) &= \mathrm{pc}_{\mathrm{soft}} + \mathrm{sg}(\mathrm{pc}_{\mathrm{hard}} - \mathrm{pc}_{\mathrm{soft}}) \\ &= \mathrm{pc}_{\mathrm{hard}}\end{aligned} \tag{6}$$

for every α and T. Every component of the handler output thus equals its hard counterpart, and as the two handlers act on the same input,

$$\Phi_{\mathrm{S}}(x) = \Phi_{\mathrm{H}}(x). \tag{7}$$

These equalities hold in float32, not merely over the reals: IEEE-754 single precision has a 24-bit significand and so represents every integer in $[-2^{24}, 2^{24}]$ exactly (IEEE 2019; Goldberg 1991), and every VCS quantity lies in this range (data bytes ≤ 255, 13-bit addresses, per-frame cycle counts of a few tens of thousands), so (5)–(7) are exact.

Induction on the trajectory. Let the two runs share the initial state and evolve by $x^{\bullet}_{t+1} = \Phi_{\bullet}(x^{\bullet}_t)$. We show $x^{\mathrm{S}}_t = x^{\mathrm{H}}_t$ for all $0 \leq t \leq N$.

Base case ($t = 0$): $x^{\mathrm{S}}_0 = x^{\mathrm{H}}_0$ by the shared initial state.

Inductive step: assume $x^{\mathrm{S}}_t = x^{\mathrm{H}}_t =: x$ for some $t < N$. Applying the one-step equality (7) to x,

$$x^{\mathrm{S}}_{t+1} = \Phi_{\mathrm{S}}(x^{\mathrm{S}}_t) = \Phi_{\mathrm{S}}(x) = \Phi_{\mathrm{H}}(x) = \Phi_{\mathrm{H}}(x^{\mathrm{H}}_t) = x^{\mathrm{H}}_{t+1}, \tag{8}$$

where the outer equalities use the inductive hypothesis and the middle one is (7). By induction $x^{\mathrm{S}}_t = x^{\mathrm{H}}_t$ for all $t \leq N$.

Gradients. The stop-gradient in (4) alters only the reverse-mode rule: the returned counter carries the derivative of $\mathrm{pc}_{\mathrm{soft}} = \mathrm{pc}_{\bar{b}} + g\,\delta$, namely $\alpha\, g(1-g)\,\delta$, which is generically nonzero where the hard branch—a step function of the flag—has zero or undefined derivative. Likewise the read (1) has Jacobian $\mathbf{1}_a$ with respect to r. The forward pass is unchanged while gradients become available. □

Theorem 2 (Temperature-limit bound)**.** *Consider the fully relaxed emulator that uses the softmax dispatch* (2) *and sigmoid branch* (3) without *the straight-through correction. Under Assumption 1, its one-step map converges to the hard map as $T \to 0$ and $\alpha \to \infty$, with per-step deviation*

$$\left\|\Phi^T_{\mathrm{S}}(x) - \Phi_{\mathrm{H}}(x)\right\|_\infty \;\leq\; C\big[(K-1)\, e^{-\Delta_{\min}/T} + e^{-\alpha m_{\min}}\big], \tag{9}$$

where C bounds the value range (bytes ≤ 255, branch offsets ≤ 256). Over N steps, with Lipschitz constant $L \geq 1$ for error propagation, the trajectory deviation is at most $\frac{L^N-1}{L-1}\, C[(K-1)e^{-\Delta_{\min}/T} + e^{-\alpha m_{\min}}] = \mathcal{O}(e^{-c/T})$ with $c = \Delta_{\min}$.

Proof. We bound the deviation of the relaxed one-step map from the hard one and then propagate it along the trajectory. Throughout, C bounds the range of any state value (bytes ≤ 255, branch offsets ≤ 256).

Dispatch term. At a step with logits ℓ and winner $k^\star$ the gap is $\Delta \geq \Delta_{\min} > 0$ by Assumption 1, so for any loser $k \neq k^\star$,

$$w_k = \frac{e^{\ell_k/T}}{\sum_j e^{\ell_j/T}} \leq \frac{e^{\ell_k/T}}{e^{\ell_{k^\star}/T}} = e^{(\ell_k - \ell_{k^\star})/T} \leq e^{-\Delta_{\min}/T}, \tag{10}$$

so the non-winning mass is $1 - w_{k^\star} = \sum_{k \neq k^\star} w_k \leq (K-1)e^{-\Delta_{\min}/T}$. As the dispatch output is the convex combination $\sum_k w_k V_k$,

$$\begin{aligned}\Big\| \textstyle\sum_k w_k V_k - V_{k^\star} \Big\|_\infty &\leq (1 - w_{k^\star}) \max_k \|V_k - V_{k^\star}\|_\infty \\ &\leq (K-1)\, e^{-\Delta_{\min}/T}\, C.\end{aligned} \tag{11}$$

Branch term. For a margin $m = |z| \geq m_{\min}$ the gate error is

$$\big|\sigma(\alpha z) - [\![z > 0]\!]\big| = \sigma(-\alpha m) \leq e^{-\alpha m_{\min}}, \tag{12}$$

so the blended counter $\mathrm{pc}_{\bar{b}} + g\,\delta$ differs from the hard target $\mathrm{pc}_{\bar{b}} + [\![z > 0]\!]\,\delta$ by

$$\big|(g - [\![z > 0]\!])\,\delta\big| \leq |\delta|\, e^{-\alpha m_{\min}} \leq C\, e^{-\alpha m_{\min}}. \tag{13}$$

Per-step bound. Adding (11) and (13) bounds the one-step deviation: for every reachable x,

$$\begin{aligned}\delta_{\mathrm{step}} &:= \left\|\Phi^T_{\mathrm{S}}(x) - \Phi_{\mathrm{H}}(x)\right\|_\infty \\ &\leq C\big[(K-1)e^{-\Delta_{\min}/T} + e^{-\alpha m_{\min}}\big],\end{aligned} \tag{14}$$

which is (9).

Propagation. Let $L \geq 1$ be a Lipschitz constant of Φ_{H} in $\|\cdot\|_\infty$ and $e_t = \|x^{T,\mathrm{S}}_t - x^{\mathrm{H}}_t\|_\infty$ the trajectory error after t steps, with $e_0 = 0$. Each step adds at most δ_{step} to the hard map applied to the accumulated error,

$$e_{t+1} \leq L\, e_t + \delta_{\mathrm{step}}, \tag{15}$$

and unrolling from $e_0 = 0$ gives

$$e_N \leq \delta_{\mathrm{step}} \sum_{i=0}^{N-1} L^i = \delta_{\mathrm{step}}\, \frac{L^N - 1}{L - 1}. \tag{16}$$

As $T \to 0$ and $\alpha \to \infty$, $\delta_{\mathrm{step}} = \mathcal{O}(e^{-\Delta_{\min}/T}) \to 0$, so the relaxed trajectory converges to the hard one, $\Phi^T_{\mathrm{S}} \to \Phi_{\mathrm{H}}$. □

3 Effect of the Relaxation Parameters on the Gradient

Theorems 1 and 2 say the hard limit ($\alpha \to \infty$ for the sigmoid branch, $T \to 0$ for the softmax select) is exact in value but degenerate in gradient. We illustrate both knobs on jutari with a single target sprite (an invader) whose horizontal placement is produced by the soft primitives. For the sigmoid branch we plot the screen-space directional-derivative saliency $\partial(\text{screen})/\partial(\text{control})$ as the sharpness α is swept (Figure 1). The main paper additionally plots the gate and its gradient directly, and shows the softmax select's temperature effect in pixel space (a sampled sprite-occupancy

heatmap). In the gradient maps the colour encodes the *sign* of the derivative—red where a pixel brightens as the control increases (the sprite arriving) and blue where it darkens (the sprite leaving)—and its intensity the magnitude, on a black background. The colour bar is in absolute units and the per-panel number is the peak relative to the row maximum.

For the branch the position is $\mathrm{pc} = (1-g)\,\mathrm{pc}_L + g\,\mathrm{pc}_R$ with $g = \sigma(\alpha z)$, so its derivative with respect to the control z is $\alpha\,\sigma(\alpha z)\,(1-\sigma(\alpha z))\,(\mathrm{pc}_R - \mathrm{pc}_L)$: a dipole, blue on the placement being vacated and red on the one being entered. The magnitude $\alpha\,\sigma(\alpha z)(1-\sigma(\alpha z))$ is *non-monotonic* in α at the fixed operating point z=0.25. A small α gives a shallow gate and hence a small slope, and a large α saturates the gate ($\sigma(\alpha z) \to 1$, so $\sigma(1-\sigma) \to 0$). The maximum is the intermediate α for which αz falls in the sigmoid's steep band, near α=6 here—α=2, 6, 20 give peak magnitudes 0.53, 1.00, 0.15, and at α=20 the gate has essentially switched and the gradient has nearly vanished (the hard-branch limit). The main paper plots this gate and its gradient directly.

The softmax select positions the sprite as the weighted sum $\sum_k w_k$ (sprite at column k) with $w = \mathrm{softmax}(\ell/T)$, so its expected screen occupancy is a blend over candidate columns whose spread grows with T. The main paper visualises this in pixel space (the sampled sprite-occupancy heatmap): a broad, smooth occupancy cloud at high T ($\sigma \approx 6$ px at T=2.0) that concentrates as T falls ($\sigma \approx 3$ px at T=0.5, 1.3 px at T=0.1) onto a single sharp sprite—the hard pick. The gradient with respect to the control follows the same arc: spread over the band at high T and collapsing to zero between pixels in the hard limit.

In every panel the forward render is the exact hard one (Theorem 1), and only the gradient changes. It is most informative at intermediate relaxation and decays to zero as the relaxation is sharpened toward the hard limit (Theorem 2). This is a qualitative study of how the parameter *values* act on the gradient, run on jutari, and the values need not be tuned for the bit-exact forward pass.

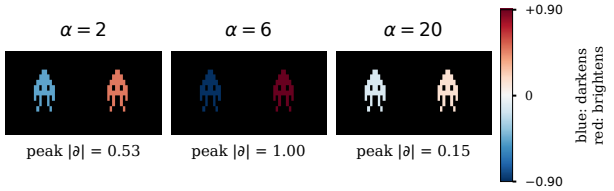

Figure 1: Soft branch: the screen-space gradient $\partial(\text{screen})/\partial z$ as the sharpness α is swept, on jutari. The target sprite's placement is a sigmoid-gated blend of a left and a right position, so the gradient is a dipole—blue where a pixel darkens (the placement being vacated) and red where it brightens (the one being entered). The colour bar is in absolute units (here $\pm$0.90) and the per-panel number is the peak relative to the maximum. The dipole is strongest at an intermediate α (here α=6) and nearly vanishes by α=20 as the gate saturates—the hard-branch limit (cf. Theorems 1 and 2).

4 Forward Bit-Exactness Under Full Relaxation

The gradient study above keeps the straight-through estimator, so the forward pass is bit-exact at any α and T (Theorem 1). To probe the temperature-limit bound (Theorem 2) empirically we instead *drop* the straight-through correction—the fully relaxed forward, where α and T act on the executed values themselves—and run it on the full jutari simulator executing the real Space Invaders ROM. Every instruction casts the sigmoid-blended program counter and the temperature-blended operand reads back to integers, so the trajectory stays bit-exact only while every such cast rounds to the value the executed (straight-through) path would produce: a large α keeps the rounded soft program counter on the hard branch target, and a small T keeps the distance-softmax read on the addressed byte.

Because each cast is a rounding, the likelihood of staying bit-exact follows from the cast margins. A crisp-flag branch with signed offset δ keeps its rounded soft program counter on the hard target exactly when $|\delta| < \tau_b(\alpha)$, $\tau_b(\alpha) = \frac{1}{2}(1+e^{\alpha})$, so the per-branch exactness $p_{\mathrm{branch}}(\alpha)$ is the fraction of executed branch offsets below τ_b. A temperature-T read at address a returns $\mathrm{round}\big(\sum_k w_k\,\mathrm{mem}[a{+}k]\big)$ with $w_k \propto e^{-|k|/T}$, and is exact when the neighbour pull $\big|\sum_{k\neq 0} w_k(\mathrm{mem}[a{+}k] - \mathrm{mem}[a])\big| < \frac{1}{2}$, giving a per-read exactness $p_{\mathrm{read}}(T)$. Treating the ρ reads and the occasional branch of an instruction as independent casts, the per-step bit-exactness likelihood is $P_{\mathrm{step}}(\alpha, T) = p_{\mathrm{read}}(T)^{\rho}\,p_{\mathrm{branch}}(\alpha)^{f_b}$, with $\rho = 2.16$ reads per instruction and branch fraction $f_b = 0.374$ measured on the executed Space Invaders trace, and the run is bit-exact only while every cast rounds correctly.

Figure 2 maps this likelihood across the whole (α, T) plane: a bit-exact corner at large α and small T, a gradual branch boundary, and a sharp temperature boundary. Table 1 zooms into the two boundaries. As the faithful measurement we report the first instruction at which the relaxed trajectory deviates from the executed one over $N = 3000$ steps ("exact" means no deviation): the game re-initialises much of its state every frame, so a diverged run can partially resynchronise and an end-of-run state match would overstate exactness. Measurement and model agree—the forward is bit-exact precisely where $P_{\mathrm{step}} = 1$, namely $\alpha \geq 6$ (so τ_b exceeds the largest offset, 122) and $T \leq 0.12$, and the first-divergence step grows as $P_{\mathrm{step}} \to 1$ (α=3, 4, 5 at T=0.12 first diverge at steps 879, 1121, 1216). The branch boundary is gradual in α while the temperature boundary is sharp: a single max-amplitude neighbour already pulls a read by $\approx 1.7 > \frac{1}{2}$, so p_{read} drops from 1 at $T \leq 0.12$ to 0.14 at $T = 0.20$. This is the empirical face of Theorem 2: exactness is recovered as $\alpha \to \infty$ and $T \to 0$. The jaxtari twin uses the identical cast-to-Int logic, so the same analysis applies.

Choosing α and T. The two bounds are *program-independent*, which is what makes the setting reliable across games. A 6502 branch displacement is at most $|\delta| \leq 127$, so $\frac{1}{2}(1+e^{\alpha}) > 127$—that is $\alpha \geq \ln(2 \cdot 127 - 1) \approx 5.5$—makes every branch round correctly, and $\alpha \geq 6$ suffices.

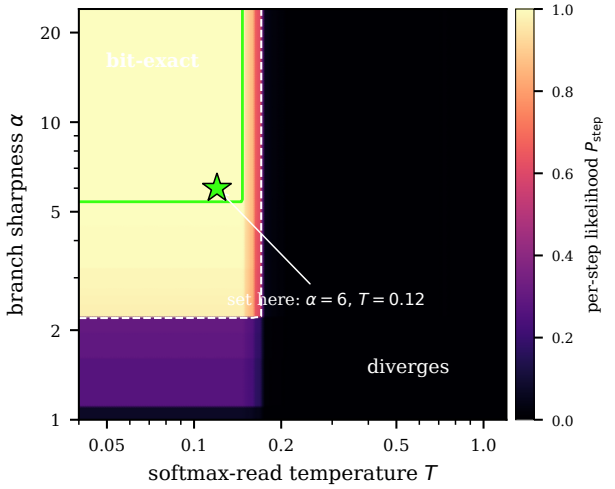

Figure 2: Overview of the per-step bit-exactness likelihood $P_{\text{step}} = p_{\text{read}}(T)^{\rho}\, p_{\text{branch}}(\alpha)^{f_b}$ of the fully relaxed forward across the (α, T) plane (Space Invaders). A bit-exact corner—bright, with the green contour marking $P_{\text{step}}{=}1$—sits at large branch sharpness α and small read temperature T, bounded by a gradual branch boundary (in α) and a sharp temperature boundary (in T); the white dashed line is $P_{\text{step}}{=}0.5$. Because P_{step} factorises, the plane is the outer combination of the two 1D profiles, and the two boundary sweeps of Table 1 are slices along the green contour's two edges. The star marks the recommended operating point ($\alpha{=}6$, $T{=}0.12$)—the corner where they cross.

A read is pulled away from its addressed byte by at most $255\,(1 - w_0) \approx 510\, e^{-1/T}$, which stays below $\frac{1}{2}$ once $T \le 1/\ln(4 \cdot 255) \approx 0.14$; the corner value $T{=}0.12$ sits comfortably inside this (worst-case pull $\approx 0.12 < \frac{1}{2}$). Both bounds depend only on the instruction set and the 8-bit byte range, not on the program, so $\alpha{=}6,\ T{=}0.12$ is bit-exact across games—verified over 6000 instructions on Pong, Breakout and Space Invaders—whereas *just outside* them the forward fails on a game-dependent subset ($\alpha{=}5$ diverges on Space Invaders but not Pong or Breakout; $T{=}0.15$ diverges on Pong and Space Invaders but not Breakout), which is exactly why the conservative bounds are needed. Within the exact region the relaxed gradient is strongest at the boundary (it vanishes as $\alpha \to \infty$, $T \to 0$; Theorem 2), so the recommended operating point is the corner of the bit-exact region, $\alpha{=}6,\ T{=}0.12$ (Figure 2): the smallest α and largest T that keep the forward bit-exact, giving the most informative gradient compatible with exactness.

Full-simulator demonstration. The per-cast model is borne out on a real 60-second rollout. We render Space Invaders beam-timed in four modes side by side (Figure 3): the exact hard run, the executed soft (straight-through) run, and the fully relaxed run at two settings. The relaxed frames come from the same cycle-accurate hard renderer driven through a default-off relaxation hook on the CPU's branch and reads, so HARD, SOFT-STE and any in-corner setting render pixel-identically. At $\alpha{=}6,\ T{=}0.14$—inside the bit-exact corner—the relaxed run stays identical to HARD for the entire rollout. At $\alpha{=}5.5,\ T{=}0.145$—just past the read boundary—it diverges, but only mildly: the game remains recognisable with a persistent, localised error (a depleted column of invaders), because the relaxation corrupts a sprite-position read rather than the control flow.

Table 1: Forward bit-exactness of the *fully relaxed* pass (soft branch without the straight-through estimator, temperature-T reads) on the full simulator running Space Invaders, sampled around the two critical boundaries. **Measured** N_{div}: the first instruction (of $N{=}3000$ from reset) at which the relaxed trajectory deviates from the executed (straight-through) one; "exact" means no deviation over the whole run. **Predicted** (cast-margin model): per-branch exactness $p_{\text{branch}}(\alpha)$, per-read exactness $p_{\text{read}}(T)$, and the per-step likelihood $P_{\text{step}} = p_{\text{read}}^{\rho}\, p_{\text{branch}}^{f_b}$ ($\rho = 2.16$, $f_b = 0.374$); the forward is bit-exact iff $P_{\text{step}} = 1$. jaxtari uses the identical cast-to-Int logic and is omitted.

		Meas.	Predicted		
α	T	N_{div}	p_{branch}	p_{read}	P_{step}
Branch boundary ($T{=}0.12$, read-exact)					
2	0.12	7	0.039	1.000	0.298
3	0.12	879	0.953	1.000	0.982
4	0.12	1121	0.988	1.000	0.996
5	0.12	1216	0.996	1.000	0.999
6	0.12	*exact*	1.000	1.000	1.000
Temperature boundary ($\alpha{=}6$, branch-exact)					
6	0.08	*exact*	1.000	1.000	1.000
6	0.10	*exact*	1.000	1.000	1.000
6	0.12	*exact*	1.000	1.000	1.000
6	0.15	803	1.000	0.981	0.959
6	0.18	2	1.000	0.223	0.039
6	0.20	2	1.000	0.144	0.015

Delayed divergence. A setting can also boot cleanly and diverge only after many frames—the Bernoulli first failure deep into the rollout. The mechanism is that the read margins are *time-varying* for RAM: a data read is exact while its neighbouring bytes are similar and crosses $\frac{1}{2}$ only once the game state evolves into a high-contrast layout, so the most-borderline cast may be one the program reaches only late. Sweeping T just below the read boundary on a long clean-boot rollout (boot exact, relaxation enabled only for the rollout) makes the first-divergence frame recede: at $\alpha{=}20$ the rollout first deviates on frame 1 for $T{=}0.1448$, on frame 194 for $T{=}0.1446$, and only on frame 2365 ($\approx$40 s of play) for $T \le 0.1444$—reproducing the game bit-for-bit for tens of seconds before a single late RAM read first rounds wrong. (Below $T{=}0.1465$, the lowest fixed ROM-read threshold, no opcode/operand fetch ever fails, so this late divergence is driven purely by an evolving RAM read.) The exact–diverge boundary is thus a per-cast threshold *spectrum* over both fixed (ROM) and state-dependent (RAM) reads, not a single cliff.

5 Implementation Effort

Figure 4 is the evidence behind the effort estimate in the main paper. Because every step of the project was committed, the commit timestamps record when work actually happened. To turn them into an active-time figure we group the commits into sessions, treating a gap of more than three hours between consecutive commits as a session boundary. Three hours is well above the in-session rhythm—the median gap between consecutive commits is about 13 minutes—and well below the multi-hour and overnight gaps that separate genuine work periods, so the partition is insensitive to the exact threshold: the active total is 106 hours at a two-hour cutoff and 162 hours at a four-hour cutoff, bracketing the 137 hours obtained at three hours.

With the three-hour boundary the 373 commits fall into 52 sessions whose durations sum to 136.8 hours, i.e. about 5.7 eight-hour working days, even though they are spread over a 29-day calendar window. The remaining $\sim$80% of the calendar span is idle and excluded. It consists mostly of stretches during which the lead programmer was travelling and without reliable internet access, which appear in the plot as the long flat segments. This active total—not the calendar span—is what we compare against the scale of the reference codebase in the main paper.

Figure 3: One frame of the 60-second full-simulator comparison (Space Invaders, beam-timed). **HARD** and **SOFT-STE** are pixel-identical (Theorem 1). **SOFT** α=6, T=0.14 is inside the bit-exact corner and stays identical to HARD for the whole rollout. **SOFT** α=5.5, T=0.145 is just past the read boundary and diverges *mildly*: the game stays recognisable but a column of invaders is missing—the localised effect of one corrupted sprite-position read, persistent over the rollout.

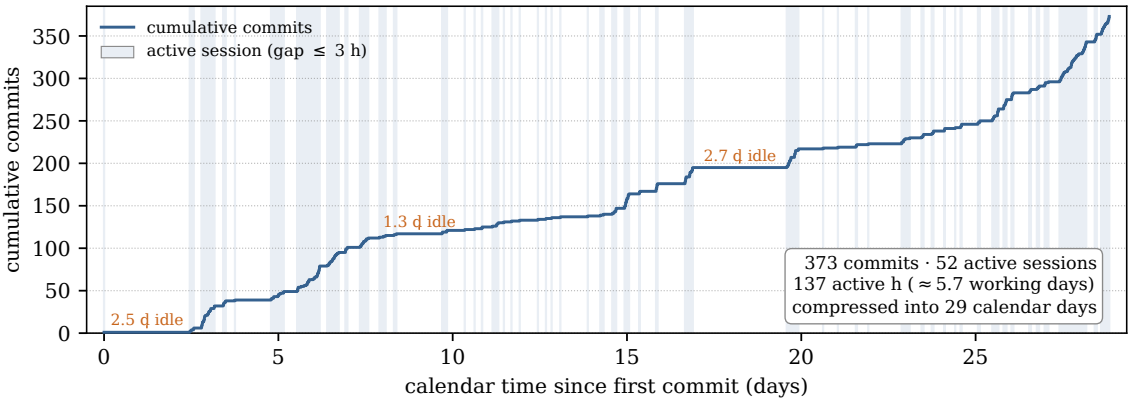

Figure 4: Implementation effort reconstructed from the version-control log. The step curve is the cumulative number of commits against calendar time, and each shaded band is an active work session (a maximal run of consecutive commits no more than three hours apart), and the long flat stretches between bands are idle gaps, the largest of which are annotated. The 373 commits form 52 active sessions totalling ∼137 hours of work—about 5.7 working days—spread across a 29-day calendar window.

References

Goldberg, D. 1991. What Every Computer Scientist Should Know About Floating-Point Arithmetic. *ACM Computing Surveys*, 23(1): 5–48.

IEEE. 2019. IEEE Standard for Floating-Point Arithmetic (IEEE Std 754-2019). IEEE Std 754-2019.